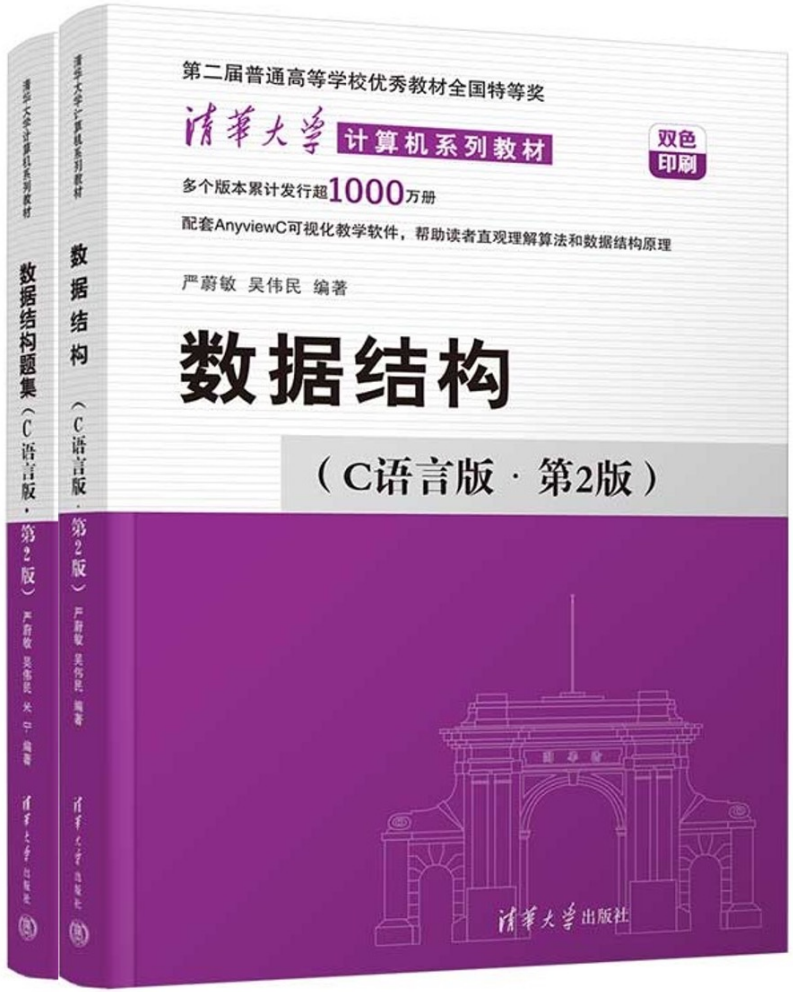


清華大學出版社
TSINGHUA UNIVERSITY PRESS

第11章

外部排序

败 者 树

我们先看胜者树记录的锦标赛决出全部名次的过程。

败者树也是一棵完全二叉树。

叶子是参与比赛的选手。关键字代表实力，值越小越强。

与胜者树不同，内部结点存放的不是胜者，而是败者的序号。

讲解算法前，直观演示图11.4(a)的败者树的初始构建，让大家对败者树及其初建有基本了解。

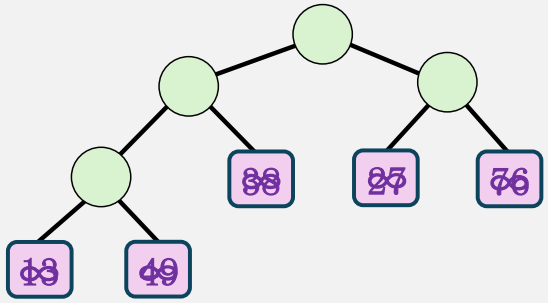
通过用败者树记录锦标赛的竞赛过程，了解了败者树的构建要点。

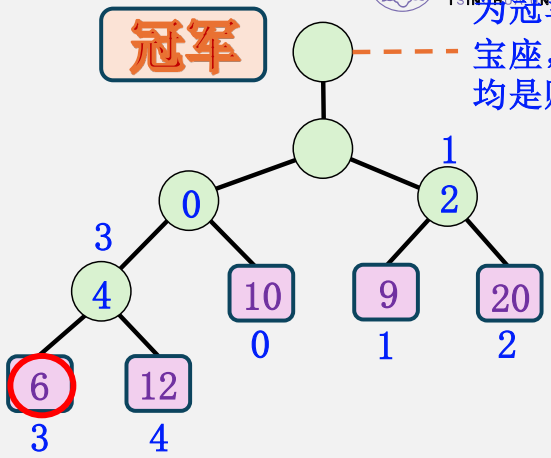
思考：

怎样利用败者树决出**全部名次**？

怎样为败者树选用合适**存储结构**？

如何设计或理解相关**算法**？





竞赛解说：

竞赛规则是：关键字小的为胜者。5位选手参赛，序号依次为0到4。

3和4号选手先赛一场。4号败，被父结点标记，而胜者3号继续向上挑战。

3号和0号比赛，0号败了，标记在父结点，3号乘胜继续向上。

另一边，1号和2号竞争，父结点标记败者2号，1号向上挑战。

3号与1号打决赛，父结点标记败者亚军1号，3号荣登冠军宝座！

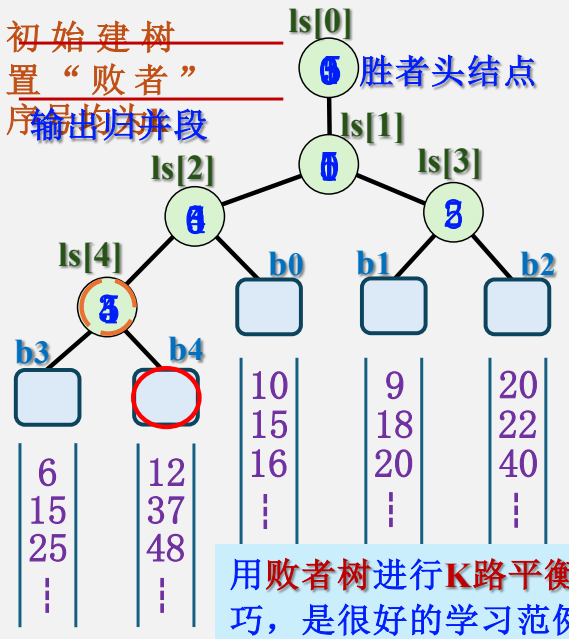
请看冠军之后名次决出过程。注意：演示的是选手序号，实际比较的是关键字。

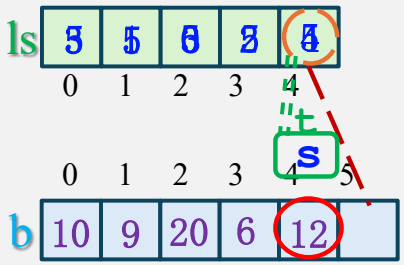
由于败者树动态保存了败者的序号，按内部结点的下标从大到小，仅沿一个叶子到根的路径即可决出下一个名次。

换成是5个队参赛，每队按关键字从小到大排序。选手晋级，后1个选手接续比赛。

冠军队的第2位选手也有可能得到亚军，甚至全队名次高于其他队的选手。

这就是用**败者树**实现**K路平衡归并**的原因。





算法11.1将各段的第1个记录读入数组b (只显示关键字)。
调用算法11.3建败者树。

用败者树进行K路平衡归并的算法设计非常精巧，是很好的学习范例。应该和求赫夫曼编码、用并查集求解等价关系、堆排序等采用沿叶子和根之间路径处理问题的算法作对比学习。

算法11.1对 k 个归并段进行平衡归并。败者树存储结构是：
用数组 b 的0至k-1单元存储 k 个叶子，每个叶子对应1个归并段，存储段中当前归并记录，下标作为序号。
用数组 ls 作为败者树内部结点部分的顺序存储结构，闲置的0号单元是增设的头结点，存储胜者序号。

构造初始败者树之后，归并过程由一个while循环完成。

```
void CreateLoserTree(LoserTree ls){
  b[k].key = MINKEY; // 最小关键字设为0
  for(i=0; i<k; ++i) ls[i] = k;
  for(i=k-1; i>=0; --i) Adjust(ls, i);
}
```
调算法11.2调整各叶子到根路径上败者位置

```
void Adjust(LoserTree ls, int s) {
  t = (s+k)/2;  // 求s叶子的父结点编号t
  while(t>0) { // 沿到根路径调整
    if(b[s].key>b[ls[t]].key)s⇔ls[t];
    t = t/2;
  }
  ls[0] = s;
}
```
若b[s]为败者，则让序号 s 和 ls[t] 交换
让 s 继续向上挑战

```
// 算法11.1的归并过程：  各归并段的结束标志
while(b[ls[0]].key!=MAXKEY) {
  q = ls[0];  // 在头结点取胜者序号
  output(b[q]); // 送胜者到输出归并段
  b[q].key=input(q); // 输入胜者后继
  Adjust(ls, q);  // 沿该后继到根路径
}             // 调整败者树，选出下一胜者
output(ls[0]);
```

11.4 置换选择排序

置换-选择排序(Replacement-Selection Sorting)是在树形选择排序的基础上得来的，它的特点是：在整个排序（得到所有初始归并段）的过程中，选择最小（或最大）关键字和输入、输出交叉或平行进行。

已知初始文件含有24个记录，关键字分别为51，49，39，46，38，29，14，61，15，30，1，48，52，3，63，27，4，13，89，24，46，58，33，76。假设内存工作区可容纳6个记录，则按上章讨论的选择排序可求得如下4个初始归并段：

RUN1：29，38，39，46，49，51
RUN2： 1，14，15， 30，48，61
RUN3： 3， 4， 13， 27，52，63
RUN4：24，33，46，58，76，89

若按置换-选择进行排序，则可求得如下3个初始归并段：

RUN1: 29，38，39，46，49，51，61
RUN2：1， 3，14，15，27，30，48，52，63，89
RUN3：4，13，24，33，46，58，76



11.4 置换选择排序 (续)

假设初始待排文件为输入文件FI，初始归并段文件为输出文件FO，内存工作区为WA，FO和WA的初始状态为空，并设内存工作区WA的容量可容纳w个记录，则置换－选择排序的操作过程为：

(1) 从FI输入w个记录到工作区WA。

(2) 从WA中选出其中关键字取最小值的记录，记为MINIMAX记录。

(3) 将MINIMAX记录输出到FO中去。

(4) 若FI不空，则从FI输入下一个记录到WA中。

(5) 从WA中所有关键字比MINIMAX记录的关键字大的记录中选出最小关键字记录，作为新的MINIMAX记录。

(6) 重复(3)～(5)，直至在WA中选不出新的MINIMAX记录为止，由此得到一个初始归并段，输出一个归并段的结束标志到FO中去。

(7) 重复(2)～(6)，直至WA为空。由此得到全部初始归并段。

11.4 置换选择排序 (续)

清華大學出版社 TSINGHUA UNIVERSITY PRESS

算法11.4是置换－选择排序的主架构，其中，

（1）调用**算法11.7**初建败者树；

（2）while循环内反复调用**算法11.5**求得每一个初始归并段。

下面观察这两个过程

```
void Replace_Selection(LoserTree ls, WorkArea wa) {
  // 算法11.4 在败者树ls和内存工作区wa上用置换-选择排序求初始归并段
  RcdType RUNEND_SYMBOL;
  RUNEND_SYMBOL.key=MAXKEY; // 段结束标志
  FI_i = FO_i = 0;          // 输入、输出计数
  Construct_Loser(ls, wa);  // 调用算法11.7 初建败者树
  rc = rmax = 1;    // rc指示当前生成的初始归并段的段号,
                    // rmax指示wa中关键字所属初始归并段的最大段号
  while (rc <= rmax) { // 用置换-选择排序求各初始归并段
    get_run(ls, wa);        // 求得一个初始归并段
    output(RUNEND_SYMBOL); // 输出段结束"记录"
    rc = wa[ls[0]].rnum;    // 设置下一段的段号
  }
}
```

11.4 置换选择排序 (续)

（1）算法11.7构建败者树的过程

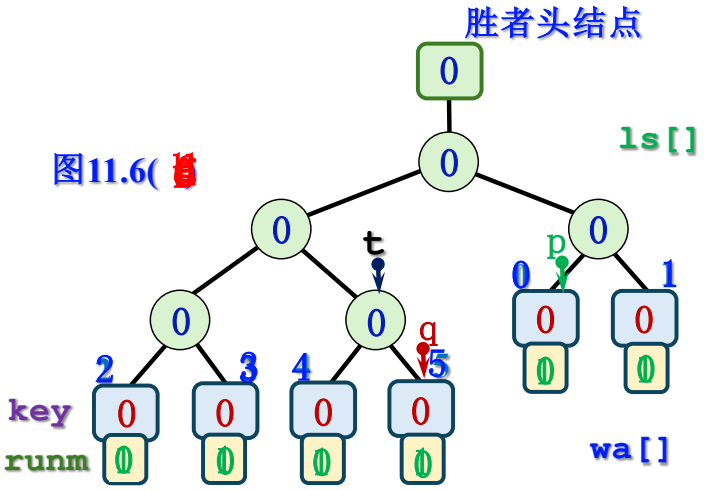
```
void Construct_Loser(LoserTree ls, WorkArea wa){
  for(i=0; i<w; ++i) // 初始化
    wa[i].runm = wa[i].key = ls[i] = 0;
  for(i=w-1; i>=0; --i) {
    wa[i].rec = inputNext();
    wa[i].key = wa[i].rec.key;
    wa[i].runm = 1;
    Select_MiniMax(ls, wa, i);
  }
}
```

算法11.7

```
void Select_MiniMax(LoserTree ls, WorkArea wa,
                    int q) {
 for(int t=(w+q)/2, p=ls[t]; t>0; t=t/2, p=ls[t])
   if(wa[p].runm<wa[q].runm ||
     wa[p].runm==wa[q].runm && wa[p].key<wa[q].key)
     q ⇔ ls[t];
 ls[0] = q;
}
```

若b[q]为败者，则让序号q和ls[t] 交换，让q继续向上挑战

算法11.6



51	49	39	46	38	29
14	61	15	30	1	48
52	3	63	…		

初始输入序列

11.4 置换选择排序 (续)

清華大學出版社 TSINGHUA UNIVERSITY PRESS

算法11.4是置换一选择排序的主架构，其中，

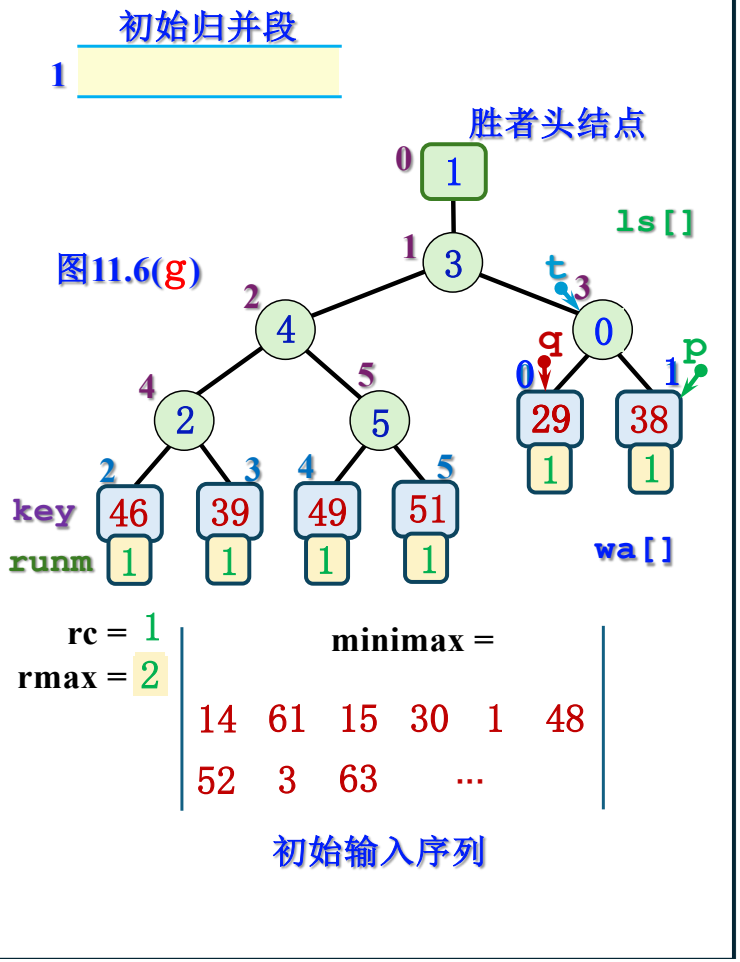
（1）调用算法11.7初建败者树；

（2）while循环内反复调用算法11.5求得每一个初始归并段。

```
void Select_MiniMax(LoserTree ls, WorkArea wa,
                    int q) {  // 算法11.6
  for (int t=(w+q)/2, p=ls[t]; t>0; t=t/2, p=ls[t])
    if (wa[p].runm<wa[q].runm ||
      wa[p].runm==wa[q].runm && wa[p].key<wa[q].key)
      q ⇔ ls[t];
  ls[0] = q;
}
```

向上调整（类似堆的调整）

```
        wa[q].rec = inputNext();      // 从输入文件读下一
        wa[q].key = wa[q].rec.key;    // 提取关键字
        if (wa[q].key < minimax) {    // 若新读入记录小于
          rmax = rc + 1;    wa[q].rnum = rmax;   // 则
        } else wa[q].rnum = rc;       // 新读入的记录属当
      }
      Select_MiniMax(ls, wa, q);      // 选择新的MINIMAX
    }
  }
```



11.4 置换选择排序 (续)

清華大學出版社 TSINGHUA UNIVERSITY PRESS

算法11.4是置换一选择排序的主架构，其中，

（1）调用**算法11.7**初建败者树；

（2）while循环内反复调用**算法11.5**求得每一个初始归并段。

```
void get_run(LoserTree ls, WorkArea wa) {  // 求得一
  while (wa[ls[0]].rnum == rc) { // 选得的MINIMAX记录
    int q = ls[0];    // q指示MINIMAX记录在wa中的位置
    minimax = wa[q].key;
    output(wa[q].rec);
    if (FI[FI_i].key == MAXKEY) { // 当读取结束时，虚
      wa[q].rnum = rmax+1;  wa[q].key = MAXKEY;
    } else {
      wa[q].rec = inputNext();      // 从输入文件读下一
      wa[q].key = wa[q].rec.key;    // 提取关键字
      if (wa[q].key < minimax) {    // 若新读入记录小于
        rmax = rc + 1;    wa[q].rnum = rmax;   // 则
      } else wa[q].rnum = rc;       // 新读入的记录属当
    }
    Select_MiniMax(ls, wa, q);      // 选择新的MINIMAX
  }
}
```

以下演示不再逐行代码跟踪

向上调整（类似堆的调整）

